Module Title

Fundamentals of Data Science

Library Resource Management System

Year

2026

Student Name: Manshi Kumari Sah

Level: 3 "D"

UEW ID: 26033159

Table of Contents

1. Introduction 3
1.1 Objectives 3
1.2 Division of Work 3
2. System Design 4
2.1 File-based Data Storage 4
2.2 Class Structure (OOP) 4
2.3 Program Flowchart 4
3. Implementation 5
3.1 data_io.py — Data Layer 5
3.2 models.py — Class Structure 7
3.3 main.py — Menu System 9
4. Testing — Sample Input and Output 12
4.1 Librarian Login and Menu 12
4.2 Failed Login and Retry 12
4.3 Student Login and Menu 12
4.4 Student Actions — Search, View Loans, Update Profile 13
4.5 A Second Student Session 14
4.6 Librarian Action — Adding a Book 15
4.7 Further Test Output 15
5. Conclusion 17

1. Introduction

This report documents the design, implementation, and testing of the Library Resource Management System, developed in Python as a Portfolio assessment for the Fundamentals of Data Science module. The system allows a library to manage its book catalogue and member loans through a menu-driven, role-based command line interface. Two types of user can log in: a Librarian, who administers the book catalogue and monitors overdue loans, and a Student, who searches the catalogue and manages their own borrowing.

The system was built collaboratively as a group of three. The project applies core concepts taught across the module, including file-based data persistence, object-oriented design through classes and inheritance, and defensive programming through exception handling.

1.1 Objectives

- Store and persist book, member, loan, and login data using plain text files.
- Provide a Librarian role able to add and remove books, update member records, and generate a report of overdue loans.
- Provide a Student role able to search the catalogue, view their own loans, and update their own profile.
- Apply object-oriented programming through a shared User class, with Librarian and Student as inheriting subclasses.
- Handle invalid input and file errors gracefully using exception handling, rather than allowing the program to crash.

1.2 Division of Work

Member	File Owned	Responsibility
Sumant	data_io.py	All file read/write functions for books, members, loans, and passwords.
Aman	models.py	User base class, authentication, and the Librarian and Student subclasses.
Manshi	main.py	Login flow, menu system, and routing between the Librarian and Student menus.

2. System Design

2.1 File-based Data Storage

The system stores all data in four plain text files, avoiding the need for an external database. Each file uses a simple comma-separated format, one record per line:

```
books.txt     ->  isbn, title, author, available (Yes/No)
members.txt   ->  username, full_name, role (Librarian/Student)
loans.txt     ->  isbn, username, borrow_date, due_date, returned (Yes/No)
passwords.txt ->  username, password
```

2.2 Class Structure (OOP)

The system's logic is organised using object-oriented principles. A base class, User, stores shared attributes (username, full_name, role) and handles authentication. Two subclasses, Librarian and Student, inherit from User and add the methods specific to each role. This avoids duplicating login logic and keeps role-specific behaviour cleanly separated.

2.3 Program Flowchart

The flowchart below illustrates the overall program logic, from login through to role-based menu handling and file updates.

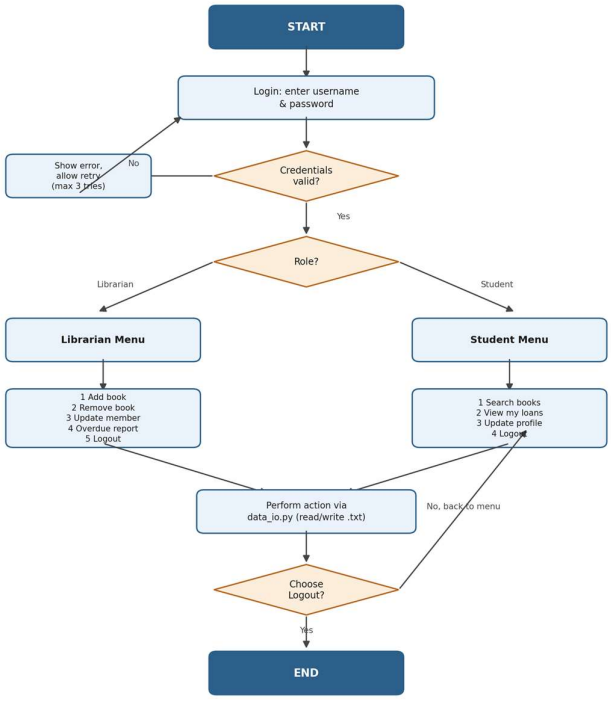


Figure 1: Overall program flow of the Library Resource Management System

3. Implementation

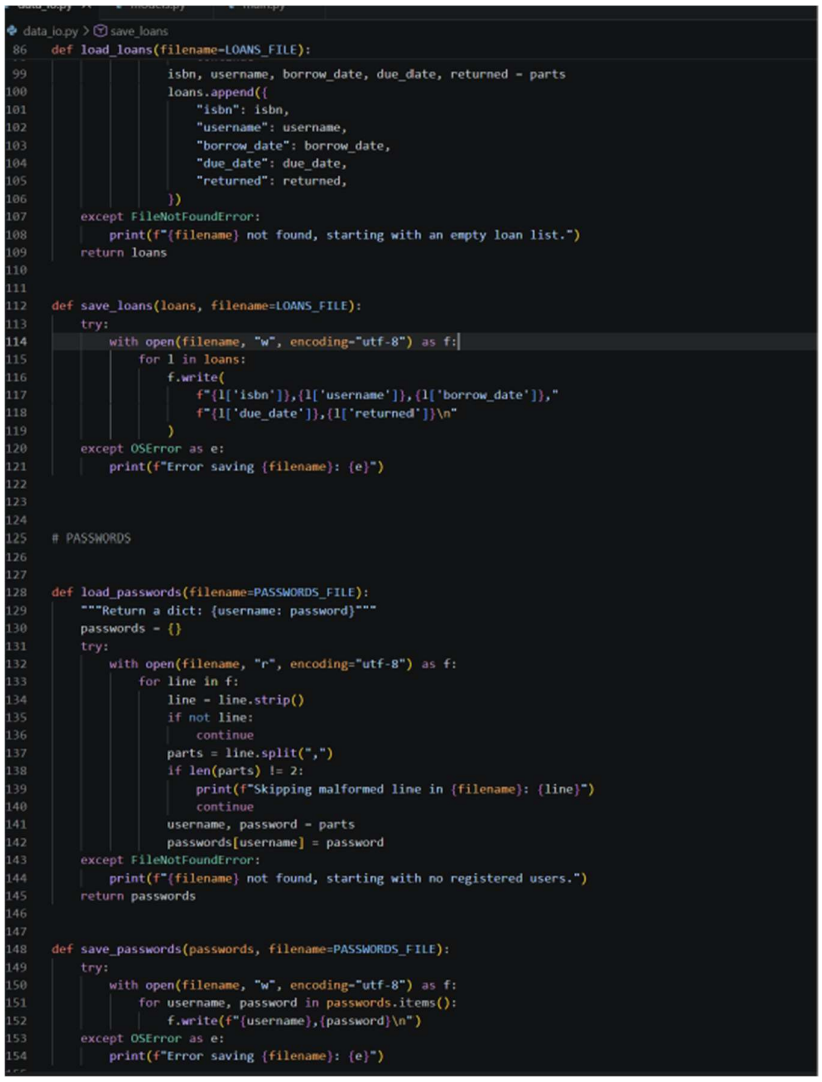
The system is split into three Python modules, each owned by one group member, plus four plain text data files. The following sections show extracts of the implemented code.

3.1 data_io.py — Data Layer

This module contains every function that reads or writes the four text files. It is the only part of the program that touches the files directly; all other modules go through it. Each load function wraps its file access in a try/except block, so a missing file starts the program with an empty list instead of crashing.




data_io.py > save_loans
MEMBERS
def load_members(filename=MEMBERS_FILE):
"""Return a list of dicts: {username, full_name, role}"""
members = []
try:
with open(filename, "r", encoding="utf-8") as f:
for line in f:
line = line.strip()
if not line:
continue
parts = line.split(",")
if len(parts) != 3:
print(f"Skipping malformed line in {filename}: {line}")
continue
username, full_name, role = parts
members.append({
"username": username,
"full_name": full_name,
"role": role,
})
except FileNotFoundError:
print(f"{filename} not found, starting with an empty member list.")
return members
def save_members(members, filename=MEMBERS_FILE):
try:
with open(filename, "w", encoding="utf-8") as f:
for m in members:
f.write(f"{m['username']},{m['full_name']},{m['role']}\n")
except OSError as e:
print(f"Error saving {filename}: {e}")
LOANS
def load_loans(filename=LOANS_FILE):
"""Return a list of dicts: {isbn, username, borrow_date, due_date, returned}"""
loans = []
try:
with open(filename, "r", encoding="utf-8") as f:
for line in f:
line = line.strip()
if not line:
continue
parts = line.split(",")
if len(parts) != 5:
print(f"Skipping malformed line in {filename}: {line}")
continue
isbn, username, borrow_date, due_date, returned = parts
loans.append({
"isbn": isbn,
"username": username,
"borrow_date": borrow_date,
"due_date": due_date,
"returned": returned,



```
data_io.py > save_loans
86  def load_loans(filename=LOANS_FILE):
99                  isbn, username, borrow_date, due_date, returned = parts
100                 loans.append({
101                     "isbn": isbn,
102                     "username": username,
103                     "borrow_date": borrow_date,
104                     "due_date": due_date,
105                     "returned": returned,
106                 })
107     except FileNotFoundError:
108         print(f"{filename} not found, starting with an empty loan list.")
109     return loans
110
111
112 def save_loans(loans, filename=LOANS_FILE):
113     try:
114         with open(filename, "w", encoding="utf-8") as f:
115             for l in loans:
116                 f.write(
117                     f"{l['isbn']},{l['username']},{l['borrow_date']},"
118                     f"{l['due_date']},{l['returned']}\n"
119                 )
120     except OSError as e:
121         print(f"Error saving {filename}: {e}")
122
123
124
125 # PASSWORDS
126
127
128 def load_passwords(filename=PASSWORDS_FILE):
129     """Return a dict: {username: password}"""
130     passwords = {}
131     try:
132         with open(filename, "r", encoding="utf-8") as f:
133             for line in f:
134                 line = line.strip()
135                 if not line:
136                     continue
137                 parts = line.split(",")
138                 if len(parts) != 2:
139                     print(f"Skipping malformed line in {filename}: {line}")
140                     continue
141                 username, password = parts
142                 passwords[username] = password
143     except FileNotFoundError:
144         print(f"{filename} not found, starting with no registered users.")
145     return passwords
146
147
148 def save_passwords(passwords, filename=PASSWORDS_FILE):
149     try:
150         with open(filename, "w", encoding="utf-8") as f:
151             for username, password in passwords.items():
152                 f.write(f"{username},{password}\n")
153     except OSError as e:
154         print(f"Error saving {filename}: {e}")
```

Figure 2: data_io.py — book loading and saving functions

3.2 models.py — Class Structure

This module defines the User base class and the Librarian and Student subclasses. The Librarian class implements add_book, remove_book, update_member, and generate_overdue_report. The Student class implements search_books, view_my_loans, and update_profile. Custom exception classes (AuthenticationError, BookNotFoundError, DuplicateBookError, MemberNotFoundError) are used to signal specific failure cases cleanly.

models.py > Librarian

```python
from datetime import datetime, date

from data_io import (
    load_books, save_books,
    load_members, save_members,
    load_loans, save_loans,
    load_passwords,
)


class BookNotFoundError(Exception):
    #Raised when a book ISBN doesn't exist in books.txt."
    pass


class DuplicateBookError(Exception):
    #Raised when trying to add a book whose ISBN already exists.
    pass


class MemberNotFoundError(Exception):
    #Raised when a username doesn't exist in members.txt.
    pass


class AuthenticationError(Exception):
    #Raised when a login attempt fails (bad username or password).
    pass


class User:

    #Base class for anyone using the system. Librarian and Student both inherit from this.


    def __init__(self, username, full_name, role):
        self.username = username
        self.full_name = full_name
        self.role = role

    def __str__(self):
        return f"{self.full_name} ({self.role}, username: {self.username})"

    @classmethod
    def authenticate(cls, username, password):

     # Check the given username/password against passwords.

        try:
            passwords = load_passwords()
            members = load_members()
        except Exception as e:
            # any unexpected file problem should surface as a clear error,
            # not a raw traceback
            raise AuthenticationError(f"Could not read login data: {e}")
```

models.py > Librarian

```python
class User:
    def authenticate(cls, username, password):

        if username not in passwords:
            raise AuthenticationError("Username not found.")

        if passwords[username] != password:
            raise AuthenticationError("Incorrect password.")

        member_record = next((m for m in members if m["username"] == username), None)
        if member_record is None:
            raise AuthenticationError("Username has no matching member record.")

        role = member_record["role"]
        full_name = member_record["full_name"]

        if role == "Librarian":
            return Librarian(username, full_name, role)
        elif role == "Student":
            return Student(username, full_name, role)
        else:
            raise AuthenticationError(f"Unknown role '{role}' for user '{username}'.")


class Librarian(User):

    def add_book(self, isbn, title, author):
        #Add a new book. Raises DuplicateBookError if the ISBN already exists.
        books = load_books()
        if any(b["isbn"] == isbn for b in books):
            raise DuplicateBookError(f"A book with ISBN {isbn} already exists.")
        books.append({
            "isbn": isbn,
            "title": title,
            "author": author,
            "available": "Yes",
        })
        save_books(books)
        return f"Added '{title}' by {author} (ISBN {isbn})."

    def remove_book(self, isbn):
    #Remove a book by ISBN. Raises BookNotFoundError if it doesn't exist.
        books = load_books()
        matching = [b for b in books if b["isbn"] == isbn]
        if not matching:
            raise BookNotFoundError(f"No book found with ISBN {isbn}.")
        books = [b for b in books if b["isbn"] != isbn]
        save_books(books)
        return f"Removed '{matching[0]['title']}' (ISBN {isbn})."

    def update_member(self, username, **fields):

    #   Update a member's record.
        members = load_members()
        member = next((m for m in members if m["username"] == username), None)
        if member is None:
```



```
models.py > Librarian
 78  class Librarian(User):
105      def update_member(self, username, **fields):
110          if member is None:
111              raise MemberNotFoundError(f"No member found with username '{username}'.")
112
113          for key, value in fields.items():
114              if key not in ("full_name", "role"):
115                  raise ValueError(f"Cannot update unknown field '{key}'.")
116              member[key] = value
117
118          save_members(members)
119          return f"Updated member '{username}': {fields}"
120
121      def generate_overdue_report(self):
122
123          #Return a list of overdue loans
124          loans = load_loans()
125          today = date.today()
126          overdue = []
127
128          for loan in loans:
129              if loan["returned"] == "Yes":
130                  continue
131              try:
132                  due = datetime.strptime(loan["due_date"], "%Y-%m-%d").date()
133              except ValueError:
134                  # a malformed date shouldn't crash the whole report
135                  print(f"Skipping loan with bad due_date: {loan}")
136                  continue
137
138              if due < today:
139                  days_overdue = (today - due).days
140                  overdue.append({**loan, "days_overdue": days_overdue})
141
142          return overdue
143
144
145  class Student(User):
146
147      def search_books(self, keyword):
148
149          # Return books whose title or author contains `keyword`.
150
151
152          books = load_books()
153          keyword = keyword.strip().lower()
154          return [
155              b for b in books
156              if keyword in b["title"].lower() or keyword in b["author"].lower()
157          ]
158
159      def view_my_loans(self):
160          #Return only the loans that belong to this student.
161          loans = load_loans()
162          return [l for l in loans if l["username"] == self.username]
163
164      def update_profile(self, **fields):
```

Figure 3: models.py

3.3 main.py — Menu System

This module contains the login loop and the two role-based menus. It calls the methods defined in models.py and wraps every menu action in try/except so invalid input or a business-logic error (e.g. duplicate ISBN) is reported to the user rather than crashing the program.

main.py > ...

```python
from models import (
    User,
    AuthenticationError,
    BookNotFoundError,
    DuplicateBookError,
    MemberNotFoundError,
)


def login():
    #Ask for username/password up to 3 times, return a User.
    for attempt in range(3):
        username = input("Username: ").strip()
        password = input("Password: ").strip()
        try:
            user = User.authenticate(username, password)
            print(f"\nWelcome, {user.full_name}! Logged in as {user.role}.\n")
            return user
        except AuthenticationError as e:
            print(f"Login failed: {e}")
            print(f"Attempts remaining: {2 - attempt}\n")
    print("Too many failed attempts. Exiting.")
    return None


def librarian_menu(librarian):
    while True:
        print("\n--- Librarian Menu ---")
        print("1. Add book")
        print("2. Remove book")
        print("3. Update member record")
        print("4. Generate overdue report")
        print("5. Logout")
        choice = input("Choose an option: ").strip()

        try:
            if choice == "1":
                isbn = input("ISBN: ").strip()
                title = input("Title: ").strip()
                author = input("Author: ").strip()
                print(librarian.add_book(isbn, title, author))

            elif choice == "2":
                isbn = input("ISBN to remove: ").strip()
                print(librarian.remove_book(isbn))

            elif choice == "3":
                username = input("Username to update: ").strip()
                new_name = input("New full name (leave blank to skip): ").strip()
                fields = {}
                if new_name:
                    fields["full_name"] = new_name
                if fields:
                    print(librarian.update_member(username, **fields))
                else:
                    print("Nothing to update.")

```



```
main.py > ...
def librarian_menu(librarian):
            elif choice == "4":
                overdue = librarian.generate_overdue_report()
                if not overdue:
                    print("No overdue books. Nice.")
                else:
                    print(f"\n{len(overdue)} overdue loan(s):")
                    for loan in overdue:
                        print(
                            f"  ISBN {loan['isbn']} | borrower: {loan['username']} "
                            f"| due: {loan['due_date']} | {loan['days_overdue']} day(s) overdue"
                        )

            elif choice == "5":
                print("Logged out.")
                break

            else:
                print("Not a valid option, try again.")

        except (BookNotFoundError, DuplicateBookError, MemberNotFoundError) as e:
            print(f"Error: {e}")
        except ValueError as e:
            print(f"Invalid input: {e}")


def student_menu(student):
    while True:
        print("\n--- Student Menu ---")
        print("1. Search books")
        print("2. View my loans")
        print("3. Update my profile")
        print("4. Logout")
        choice = input("Choose an option: ").strip()

        try:
            if choice == "1":
                keyword = input("Search keyword (title or author): ").strip()
                results = student.search_books(keyword)
                if not results:
                    print("No books matched.")
                else:
                    for b in results:
                        print(f"  {b['title']} by {b['author']} \u2014 {b['available']}")

            elif choice == "2":
                loans = student.view_my_loans()
                if not loans:
                    print("You have no loans on record.")
                else:
                    for l in loans:
                        status = "Returned" if l["returned"] == "Yes" else "Not returned"
                        print(f"  ISBN {l['isbn']} | due {l['due_date']} | {status}")

            elif choice == "3":
                new_name = input("New full name: ").strip()
                if new_name:
```

Figure 4: main.py — student menu handling and program entry point

4. Testing — Sample Input and Output

The system was tested end to end by running main.py and exercising every menu option for both roles. The screenshots below demonstrate the functionality, including successful logins, a failed login with retry, and role-specific actions.

4.1 Librarian Login and Menu

Sample input: username manshi, password librarian123.

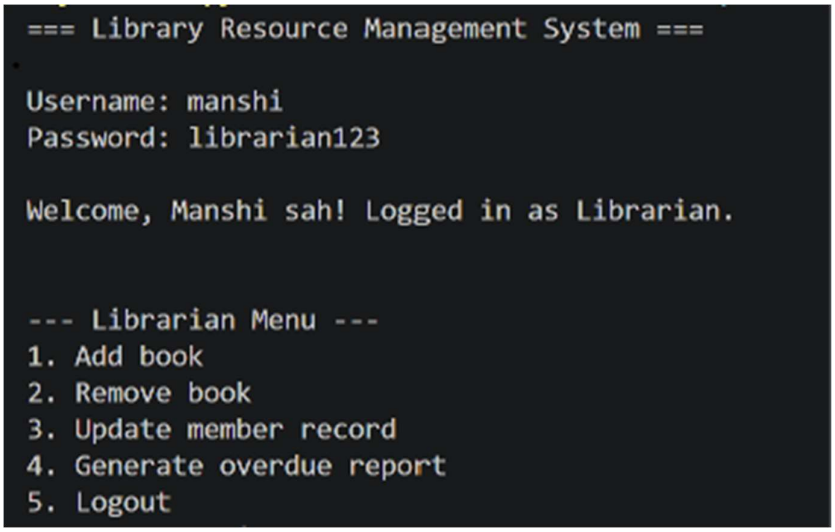


Figure 5: Successful Librarian login, showing the Librarian menu

4.2 Failed Login and Retry

Sample input: username manshi, incorrect password. Expected output: an error message and a retry prompt, without the program crashing.

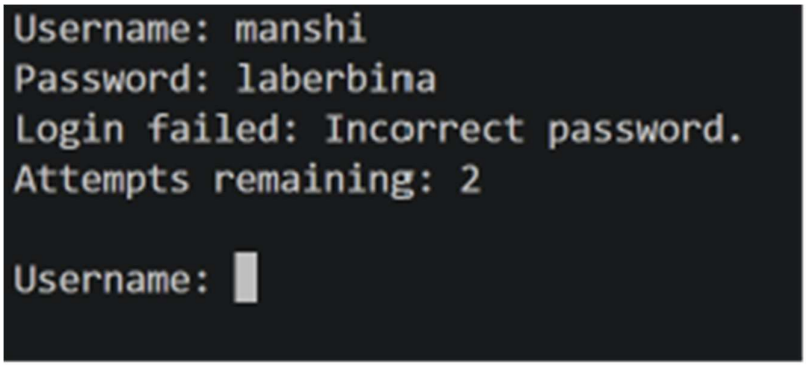


Figure 6: Incorrect password handled with a clear error and retry

4.3 Student Login and Menu

Sample input: username sumant, password student123.

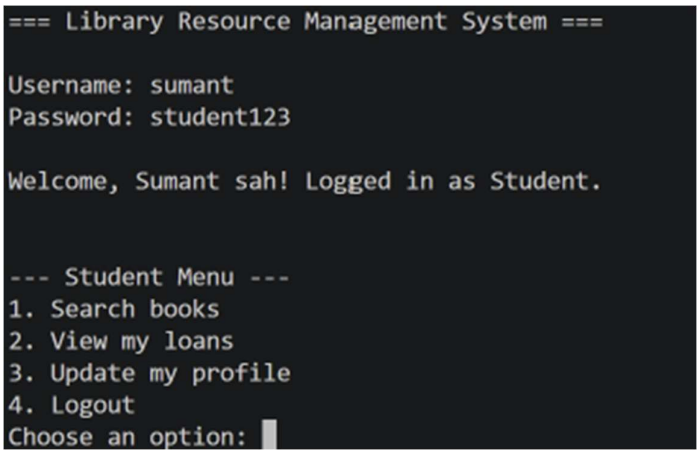


Figure 7: Successful Student login, showing the Student menu

4.4 Student Actions — Search, View Loans, Update Profile

Sample input/output for Sumant's session: searching the keyword 'FODS' returned no result on title but matched on author ('FODS by prasant sir'); viewing loans correctly showed only the ISBN borrowed by sumant with its due date; updating the profile changed the full name to 'Sumant'.



Figure 8: Student menu — search, view loans, update profile, and logout

4.5 A Second Student Session

Sample input/output for Aman's session: searching 'Introduction to Algorithms' correctly matched the corresponding book; viewing loans correctly reported no loans on record, since aman had no active loan in loans.txt at the time of testing; the profile name was updated to 'Aman'.

```
Username: aman
Password: student123

Welcome, aman mahato! Logged in as Student.


--- Student Menu ---
1. Search books
2. View my loans
3. Update my profile
4. Logout
Choose an option: 1
Search keyword (title or author): Introduction to Algorithms
  Introduction to Algorithms by Thomas — Yes

--- Student Menu ---
1. Search books
2. View my loans
3. Update my profile
4. Logout
Choose an option: 2
You have no loans on record.

--- Student Menu ---
1. Search books
2. View my loans
3. Update my profile
4. Logout
Choose an option: 3
New full name: Aman
Profile updated: {'full_name': 'Aman'}

--- Student Menu ---
1. Search books
2. View my loans
3. Update my profile
4. Logout
Choose an option: 4
Logged out.
PS C:\Users\asus\Desktop\Fods\pdf\Portfolio-manshi kumari sah -uwe id 26033159>
```

Figure 9: Student menu session for aman, showing an empty loan list

4.6 Librarian Action — Adding a Book

Sample input: ISBN 187, Title 'english', Author 'sulochana nepal'. Expected output: confirmation that the book was added.

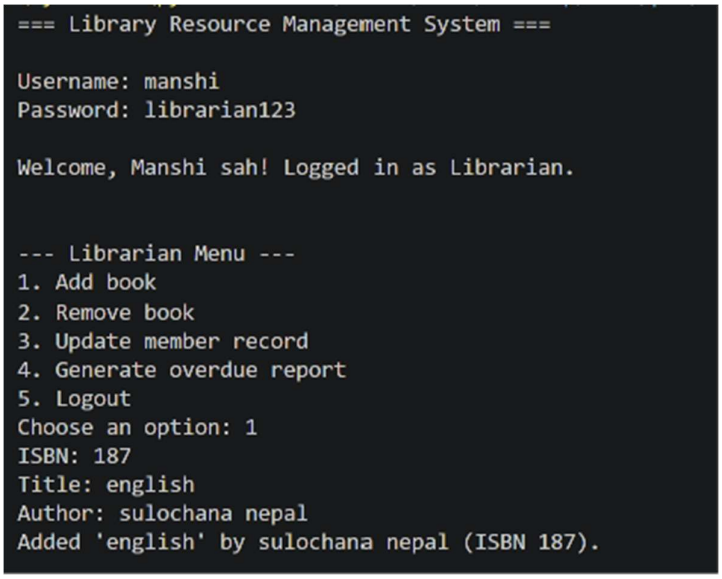


Figure 10: Librarian adding a new book to the catalogue

4.7 Further Test Output

The following additional screenshots were captured during testing to further evidence correct behaviour across the menu options.

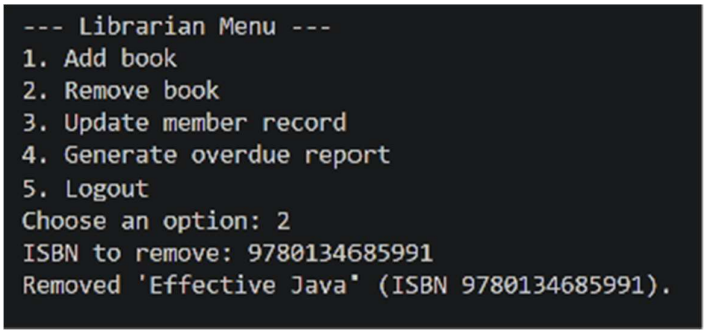


Figure 11: Additional Librarian menu action output

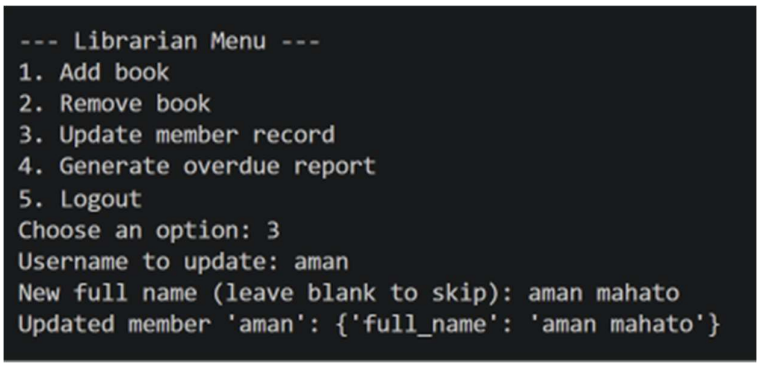


Figure 12: Additional Librarian menu action output

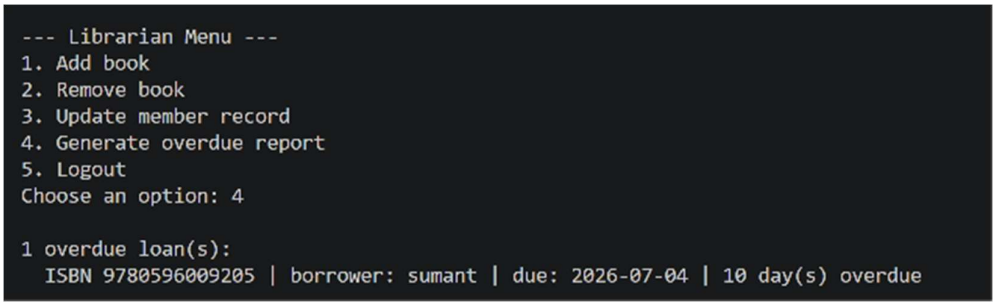


Figure 13: Additional Librarian menu action output



Figure 14: Additional Librarian menu action output

5. Conclusion

The Library Resource Management System successfully meets the requirements set out in the assessment brief. All four data files are implemented and correctly persist data between runs. The Librarian role supports adding and removing books, updating member records, and generating an overdue report. The Student role supports searching the catalogue, viewing personal loans, and updating a profile. Object-oriented design, through the User, Librarian, and Student classes, avoids duplicated logic, and exception handling throughout the program prevents crashes on invalid input or missing data.

Testing confirmed that all menu options behave as expected for both roles, and that error conditions — an incorrect password, a duplicate ISBN, or a missing book — are handled gracefully rather than causing the program to fail.